



CourseCube®
(Formerly Java Learning Center)

MicroServices

Study Guide

Module 9

Developing BookStore –V.1.6
(Cloud Config and Cloud Bus)

Author
Srinivas Dande





CourseCube®
(Formerly Java Learning Center)



Cloud Config Server and Client

- ✓ Managing application configuration is critical for microservices running in the cloud because microservice instances need to be launched quickly with minimal human intervention.
- ✓ If there is a human intervention every time to manually configure service then an unexpected outage and a lag-time in responding to scalability challenges with the application.
- ✓ To avoid this, Application configuration shouldn't be deployed with the service instance. Instead, configuration information should either be passed to the starting service as environment variables or read from a centralized repository when the service starts.
- ✓ Centralize your application configuration into Single repository as possible because a cloud-based application might literally have hundreds of services, it's critical to minimize the number of different repositories used to hold configuration information.
- ✓ When a microservice instance comes up, it's going to call a service endpoint to read its configuration information Centralized Point
- ✓ When a configuration data changes are made, then services that use that application configuration data must be notified about that change and refresh their copy of the application data.

Solution:

- ✓ **Spring Cloud Config** is Spring's client/server approach for storing and serving distributed configurations across multiple applications and environments.
- ✓ This configuration store is ideally versioned under Git version control and can be modified at application runtime.
- ✓ While it fits very well in Spring applications using all the supported configuration file formats together with constructs like Environment, PropertySource or @Value

Key Components:

- GIT Repository
- Cloud Config Server
- Cloud Config Config
- Cloud Bus



Initial Project Setup

Working Steps:

- 1) Copy **Hystrix-Demo** workspace and paste as **Cloud-Config-Demo**
- 2) In this Copied workspace, You have 3 apps
 - * BookSearchMS
 - * BookStoreWeb
 - * MyEurekaServer
- 3) Update **MyEurekaServer** (As given in Page No: 371,372)
- 4) Update **BookSearchMS** (As given in Page No: 373,374,375,376)
- 5) Update **BookStoreWeb** (As given in Page No: 377,378,379,380,381)
- 6) Start all 3 Apps
 - * Start MyEurekaServer
 - * Start BookStoreWeb
 - * Start BookSearchMS
- 7) Hit the URL as follows
`http://localhost:5000/mybook`
- 8) Stop All 3 Apps
 - * Start MyEurekaServer
 - * Start BookStoreWeb
 - * Start BookSearchMS



MyEurekaServer – Files Required

1. MyEurekaServerApplication.java	2. application.properties
3. pom.xml	

1. MyEurekaServerApplication.java

```
package com.jlcindia.myeurekaserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableEurekaServer
public class MyEurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyEurekaServerApplication.class, args);
    }
}
```

2. application.properties

```
spring.application.name=MyEurekaServer
server.port=8761

eureka.client.register-with-eureka=false
eureka.client.fetchRegistry= false
```

3. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.myeurekaserver</groupId>
<artifactId>MyEurekaServer</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>MyEurekaServer</name>
<description>Eureka - Service Registry and Discovery Server</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>2020.0.4</spring-cloud.version>
</properties>
```



```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.6</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



BookSearchMS – Files Required

1. BookSearchApplication.java	2. BookSearchController.java
3. BookInfo.java	4. application.properties
5. pom.xml	

1. BookSearchApplication.java

```
package com.jlcindia.booksearch;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableEurekaClient
public class BookSearchApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookSearchApplication.class, args);
    }
}
```

2. BookSearchController.java

```
package com.jlcindia.booksearch;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class BookSearchController {

    @Value("${server.port}")
    String searchPort;

    @Value("${jlc.search.message}")
    String searchMessage;
}
```



```
@GetMapping("/jlcbook")
public BookInfo getBook() {
    System.out.println("---BookSearchController---getBook()---");

    BookInfo bookInfo=new BookInfo();

    bookInfo.setBookId(101);
    bookInfo.setBookName("Master Spring MicroServices");
    bookInfo.setAuthor("Srinivas Dande");

    bookInfo.setSearchPort(searchPort);
    bookInfo.setSearchMessage(searchMessage);

    return bookInfo;
}
}
```

3. BookInfo.java

```
package com.jlcindia.booksearch;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class BookInfo {
    private Integer bookId;
    private String bookName;
    private String author;
    private String searchPort;
    private String searchMessage;

    //Constructors
    //Setters and Getters
}
```

4. application.properties

```
spring.application.name=MyBookSearchMS

server.port=8000

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true
```



```
management.endpoints.web.exposure.include=*  
management.endpoint.health.show-details=always
```

```
jlc.search.message = I am from BookSearch !!!
```

5. pom.xml

```
<project ...>  
<modelVersion>4.0.0</modelVersion>  
  
<groupId>com.jlcindia.booksearch</groupId>  
<artifactId>BookSearchMS</artifactId>  
<version>1.0.SNAPSHOT</version>  
  
<packaging>jar</packaging>  
  
<name>MyBookSearchMS</name>  
<url>https://www.jlcindia.com</url>  
<description>This is BookSeach MS developed by Srinivas Dande</description>  
  
<properties>  
<java.version>1.8</java.version>  
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>  
</properties>  
  
<parent>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-parent</artifactId>  
<version>2.3.3.RELEASE</version>  
</parent>  
  
<dependencies>  
  
<dependency>  
<groupId>org.springframework.cloud</groupId>  
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>  
</dependency>  
  
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-actuator</artifactId>  
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```



BookStoreWeb – Files Required

1. BookStoreApplication.java	2. BookSearchProxy.java
3. BookStoreController.java	4. BookInfo.java
5. application.properties	6. pom.xml

1. BookStoreApplication.java

```
package com.jlcindia.bookstoreweb;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
import org.springframework.cloud.openfeign.EnableFeignClients;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableFeignClients
@EnableEurekaClient
public class BookStoreApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookStoreApplication.class, args);
    }
}
```

2. BookSearchProxy.java

```
package com.jlcindia.bookstoreweb;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@FeignClient("MyBookSearchMS")
public interface BookSearchProxy {

    @GetMapping("/jlcbook")
    public BookInfo getBook();
}
```



3. BookStoreController.java

```
package com.jlcindia.bookstoreweb;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class BookStoreController {

    @Autowired
    BookSearchProxy bookSearchProxy;

    @Value("${server.port}")
    String storePort;

    @Value("${jlc.store.message}")
    String storeMessage;

    @GetMapping("/mybook")
    public BookInfo getBook() {
        System.out.println("---BookStoreController---getBook()---");

        BookInfo bookInfo = bookSearchProxy.getBook();

        bookInfo.setStorePort(storePort);
        bookInfo.setStoreMessage(storeMessage);

        return bookInfo;
    }
}
```



4. BookInfo.java

```
package com.jlcindia.bookstoreweb;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

public class BookInfo {
    private Integer bookId;
    private String bookName;
    private String author;
    private String searchPort;
    private String searchMessage;
    private String storePort;
    private String storeMessage;

    //Constructors
    //Setters and Getters
}
```

5. application.properties

```
spring.application.name=MyBookStoreWeb

server.port=5000

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

jlc.store.message = I am BookStoreWeb !!!
```

5. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.bookstoreweb</groupId>
<artifactId>BookStoreWeb</artifactId>
<version>1.0.SNAPSHOT</version>
<packaging>jar</packaging>
```



```
<name>MyBookStoreWeb</name>
<url>https://www.jlcindia.com</url>
<description>This is BookStoreWeb developed by Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
```



```
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```



Cloud Config Server and Client

Working Steps:

A) Set Up Your MicroServices

- 1) Copy Hystix-Demo workspace and paste as Cloud-Config-Demo
- 2) In this Copied workspace, You have 3 apps
 - * BookSearchMS
 - * BookStoreWeb
 - * MyEurekaServer
- 3) Update **MyEurekaServer** (As given in **Page No: 371,372**)
- 4) Update **BookSearchMS** (As given in **Page No: 373,374,375,376**)
- 5) Update **BookStoreWeb** (As given in **Page No: 377,378,379,380,381**)
- 6) Start the Apps
 - * Start MyEurekaServer
 - * Start BookStoreWeb
 - * Start BookSearchMS
- 7) Hit the URL as follows
http://localhost:5000/mybook

B) Setup Git Repository

- 8) Install the GIT
- 9) Create the folder structure required
E:\mybookstore\myjlc-repo>
- 10) Go to that folder in the command line and run the following commands

```
E:\mybookstore\myjlc-repo>git config --global user.email srinivas@coursecube.com
E:\mybookstore\myjlc-repo>git config --global user.name "Srinivas Dande"
E:\mybookstore\myjlc-repo>git init .
```

C) Update BookSearchMS

- 11) Create the Property File for **BookSearchMS** as **MyBookSearchMS.properties** under GIT Repository
- 12) Copy all the Properties related to **BookSearchMS** from **application.properties** into **MyBookSearchMS.properties**
- 13) Run the Following Git Commands

```
git add .  
git commit -m "Update #1 by Srinivas Dande"
```
- 14) Rename the **application.properties** of **BookSearchMS** as **bootstrap.properties**

#bootstrap.properties should have the following properties

```
spring.application.name=MyBookSearchMS  
server.port=8000  
spring.cloud.config.uri=http://localhost:2345
```

- 15) Add the **Cloud Config Client** dependency to the pom.xml file

```
<dependency>  
<groupId>org.springframework.cloud</groupId>  
<artifactId>spring-cloud-starter-config</artifactId>  
</dependency>
```

D) Update BookStoreWeb

- 16) Create the Property File for **BookStoreWeb** as **MyBookStoreWeb.properties** under GIT Repository
- 17) Copy all the Properties related to **BookStoreWeb** from **application.properties** into **MyBookStoreWeb.properties**
- 18) Run the Following Git Commands

```
git add .  
git commit -m "Update #2 by Srinivas Dande"
```



19) Rename the **application.properties** of **BookStoreWeb** as **bootstrap.properties**

#bootstrap.properties should have the following properties

```
spring.application.name= MyBookStoreWeb  
server.port=5000  
spring.cloud.config.uri=http://localhost:2345
```

20) Add the **Cloud Config Client** dependency to the pom.xml file

```
<dependency>  
<groupId>org.springframework.cloud</groupId>  
<artifactId>spring-cloud-starter-config</artifactId>  
</dependency>
```

E) Setup the Config Server

21) Copy MyEurekaServer as **MyConfigServer**

22) Update **pom.xml**

```
<dependency>  
<groupId>org.springframework.cloud</groupId>  
<artifactId>spring-cloud-config-server</artifactId>  
</dependency>
```

23) Rename the **application.properties** to **bootstrap.properties**

24) Add the following properties in **bootstrap.properties** of **MyConfigServer**

```
spring.application.name=MyConfigServer  
server.port=2345  
spring.cloud.config.server.git.uri = E:\\mybookstore\\myjlc-repo
```

25) Rename the Package as **com.jlcindia.myconfigserver**

26) Rename Boot main class as **MyConfigServerApplication**



27) Add the @EnableConfigServer annotation on the class.

```
@SpringBootApplication
@EnableConfigServer
public class MyConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyConfigServerApplication.class, args);
    }
}
```

F) Test Config Server

28) Start MyConfigServer.

29) Open the following link to see whether the server is running.

<http://localhost:2345/actuator/health>

If everything is fine, this will display health of config server.

30) Check the following URL to see the properties of MicroServices

<http://localhost:2345/MyBookStoreWeb/default/master>

<http://localhost:2345/MyBookSearchMS/default/master>

G) Test the Complete Eco-System

31) Start All the Apps

- * Start MyEurekaServer **(8761)**
- * Start MyCinfigServer **(2345)**
- * Start **1 Instance** of BookStoreWeb **(5000)**
- * Start **3 Instances** of BookSearchMS **(8000,8100,8200)**

32) Hit the URL : **<http://localhost:5000/mybook>**

33) You can observe that both the MicroServices will be started successfully by reading properties from ConfigServer.

Now MicroServices, ConfigServer and GIT are in Sync and Running Successfully.



Q) How the Configuration Data updated in GIT will be reflected to MicroServices:

Ans:

- 1) Re-Start the Servers (Bad Idea)
- 2) Using **refresh end-point** of actuator (OK OK)
- 3) Using **Cloud Bus** (The Best)

H) Using refresh end-point of actuator

Working Steps

34) Mark the **BookSearchController** with **@RefreshScope** annotation

```
@RestController
@RefreshScope
public class BookSearchController {
    ....
}
```

35) Mark the **BookStoreController** with **@RefreshScope** annotation

```
@RestController
@RefreshScope
public class BookStoreController {
    ....
}
```

@RefreshScope annotation has to be added at the class level to allow properties to be refreshed when there is a change.

36) Change the Property in **MyBookSearchMS.properties**

```
jlc.search.message = BookSearch - Default Message - Update 1.0
```

37) Change the Property in **MyBookStoreWeb.properties**

```
jlc.store.message = BookSearch - Default Message – First Update
```



38) Run the Following Git Commands

```
git add .
```

```
git commit -m "Update #3 by Srinivas Dande"
```

39) Ping each Instance of each MicroService with **Post request** with following URL's

```
http://localhost:5000/actuator/refresh
```

```
http://localhost:8000/actuator/refresh
```

```
http://localhost:8100/actuator/refresh
```

```
http://localhost:8200/actuator/refresh
```